

CTS SRE

Chaos Engineering Validation – 1 & 2

EMPOWERING High Performance
Technology Teams

Agenda for the Day

- 1 **Principles of Chaos Engineering** – Hypothesis-driven testing, failure injection
- 2 **Pre-requisites for Chaos Validation** – Monitoring & test environment setup
- 3 **Running Controlled Chaos Experiments** – Measuring system behavior
- 4 **Blast Radius Control in Chaos Engineering** – Limiting impact and expanding experiments
- 5 **Targeted Experimentation** – Resource isolation and dependency mapping
- 6 **Rollback and Safe State Validation** – Ensuring system recovery

Principles of Chaos Engineering

◆ **Chaos Engineering** is the discipline of experimenting on a distributed system to **build confidence in its resilience**.

◆ **Key Principles:**

✓ **Hypothesis-Driven Testing:** Define expectations before injecting failures.

✓ **Failure Injection:** Introduce disruptions in a controlled manner.

✓ **Measuring Impact:** Observe real-time behavior of applications and infrastructure.

✎ **Example:**

Hypothesis: **"If a database node fails, our system should automatically reroute traffic to replicas within 5 seconds."**

Test: Terminate a database node and measure system recovery time.

Running Controlled Chaos Experiments

Chaos Experiment Workflow

- 1 **Define Hypothesis** → Example: "System should recover from API failure in 3 seconds."
 - 2 **Select Chaos Tool** → Use **Gremlin, AWS Fault Injection Simulator (FIS)**
 - 3 **Inject Failure** → Simulate node termination, latency spikes, or resource exhaustion.
 - 4 **Monitor Behavior** → Observe system logs, alerts, and dashboards.
 - 5 **Analyze Impact** → Compare baseline vs. post-experiment performance.
-  **Activity:** Run a **CPU spike experiment on an EC2 instance** and measure response time degradation.

Blast Radius Control in Chaos Engineering



What is Blast Radius?

The extent of impact caused by a chaos experiment on the system.
Controlled chaos experiments start small and expand progressively.



Steps to Control Blast Radius:

1. Isolate Failures to Specific Services → Tag affected resources.
2. Use Traffic Mirroring → Simulate real-world failures with minimal impact.
3. Gradually Expand the Experiment → Move from a single instance to a full cluster test.



Example:

Start by terminating **1 database node**, then expand to **an entire availability zone**.



Activity: Define the **blast radius for a microservice failure test**.

Targeted Experimentation



Why Target Specific Services?

- Helps test system resilience without causing major disruptions.
- Prevents unintended cascading failures.



Techniques for Targeted Chaos Experiments:

- 1 **Tagging Resources** → Apply labels to isolate workloads (e.g., `env=chaos-test`).
 - 2 **Dependency Mapping** → Identify critical dependencies using **AWS X-Ray** or **Jaeger**.
 - 3 **Gradual Expansion** → Move from **single-node failures** → **service failures** → **region-wide failures**.
-  **Activity:** Identify and **tag a Kubernetes pod** for targeted failure injection.

Rollback and Safe State Validation



Why is Rollback Important?

- Ensures the system can return to a stable state after a failure.
- Helps maintain high availability and fault tolerance.



Key Rollback Strategies:

- 1 **Automated Rollbacks** → Use **feature flags** to disable faulty deployments.
- 2 **Circuit Breakers** → Prevent cascading failures with tools like **Hystrix or Istio**.
- 3 **Health Checks & Auto-Healing** → Configure **Kubernetes Liveness Probes** to restart failed services.



Example:

If a **database experiment causes latency**, ensure queries failover to **read replicas**.



Activity: Implement an **auto-recovery mechanism in Kubernetes** using Liveness Probes.

Hands on Exercises

Labs

- <https://github.com/AshishSharmaPrivate/ChaosLab/tree/main/Chaos-Engineering-Validation>

Overall Summary

-  **Chaos Engineering validates system resilience through structured failure injection.**
 -  **Blast Radius Control minimizes unintended disruptions.**
 -  **Targeted experimentation ensures critical services remain unaffected.**
 -  **Rollback & self-healing mechanisms maintain continuous availability.**
-  **Q&A & Discussion**

Thank You!
StackRoute
(An NIIT Venture)

